
Geant4講習会受講者ためのC++入門

C++11 以降対応

Geant4講習会資料：2025 改訂版



大学共同利用機関法人
高エネルギー加速器研究機構

本資料に関する注意

- 本資料の知的所有権は、高エネルギー加速器研究機構およびGeant4 collaborationが有します
- 以下のすべての条件を満たす場合に限り無料で利用することを許諾します
 - 学校、大学、公的研究機関等における教育および非軍事目的の研究開発のための利用であること
 - ・ Geant4の開発者はいかなる軍事関連目的へのGeant4の利用を拒否します
 - このページを含むすべてのページをオリジナルのまま利用すること
 - ・ 一部を抜き出して配布したり利用してはいけません
 - 誤字や間違いと疑われる点があれば報告する義務を負うこと
- 商業的な目的での利用、出版、電子ファイルの公開は許可なく行えません

はじめに

■ 講義の目標

- プログラミングの経験はあるが、C++は複雑で良く分からんという受講者が対象
- C++で作られたGeant4を使用するのに最低限必要な言語知識を解説

- | | |
|------------------|-----------------------|
| 1) C++プログラムの基本構造 | 4) コンストラクタとオブジェクト存続期間 |
| 2) オブジェクトとは | 5) 継承とは |
| 3) クラスとは | 6) 動的多態性と仮想関数とは |

[注] もちろん、これらの知識を短時間の講義で学ぶことは不可能。この講義で上記の用語の概念を知り、Hands-onプログラムを読み進め、C++で分からないところをInteractive AIに聞いて理解して行くのが、実用的なC++習得方法であろう。

■ Geant4がC++言語を使用している理由

- Geant4は**オブジェクト指向設計**を基礎にデザインした（ほぼ30年前）
- デザインを実装する時点で処理・実行が高速な**オブジェクト指向言語**はC++が唯一だった
- C++は長い歴史を持つ**オブジェクト指向言語** — Geant4はC++の歴史とともに歩んできた

■ Modern C++ (C++11以降)と Legacy C++ (C++11より前)

- Geant4は歴史の長いプログラム — Legacy C++で構成されている部分が多い
- Modern C++は上位互換性を保証しているので、ユーザは相違を意識する必要なし
- Geant4ユーザは自分のコードにModern C++を自由に使用できる

C++プログラムの基本構造

C++プログラムの基本構造 — 'Hello World!'から学ぶ

```
// プログラム名: HelloWorld.cc
```

```
#include <iostream>
```

```
int main()  
{
```

```
    std::cout << "Hello, world!"  
    << std::endl;
```

```
    return 0;
```

```
}
```

← コメント行は//から始める（行末までがコメント）

- `/* comment */` もコメント

← 使用するライブラリは、`#include`で事前に宣言ファイル(header)を取込む

- `#include<...>`は標準ライブラリ、`#include"..."`はプログラム内のファイル取込
- `iostream`はC++の標準入出力ライブラリ: `cout`、`endl`が使用可能となる
- 通常、関数定義のブロック`{ }`の外におく
- 使用前に宣言が必要な変数名、関数名、クラス名もここにおく(前方宣言)

← プログラム処理は`main`関数から開始

- `main`はアプリケーション全体で一つだけ存在
- 帰り値は整数型(`int`): `zero`は正常終了、それ以外は異常終了
- 関数の本体は`{ }`内に書く

← Stringの端末への出力

- `cout`(標準出力ストリーム)、`<<`(出力演算子)を使う
- `::`(スコープ演算子)使い`std::cout`で標準ライブラリの`cout`を使うことを明示
- `endl`は改行を端末に出力 - `cout`と同様に`'std::'`および`<<`を使う

← 関数の帰り値は`return`で指定

- `main`関数では帰り値を整数にする: `"= 0"` 正常終了、`"= 0"` 異常終了
- C++の文の後にはセミicolon `;`を書く

[注] スライドのC++コードをcopy/pasteして使用するのは危険 — 特殊文字が文字化けする可能性あり

C++プログラムの基本構造 – 続き

■ 標準ライブラリとは

- C++規約で定義されるライブラリで、C++言語体系と一体： C++コンパイラに包含される
- 標準入出力を含む広範囲な言語機能を網羅： 後のhands-onで解説
- ライブラリ・コンポーネントは名前衝突回避のため”std”名前空間のもとで使う： ”std”は苗字のようなもの
- 名前空間の指定にはスコープ演算子’::’を使う： 例) std::cout

■ 名前空間の省略

- ライブラリの使用で、名前衝突の心配がない場合は以下の様に”std::”を省略できる

```
#include <iostream>

int main()
{
    using namespace std;
    cout << "Hello, world!" << endl;
    return 0;
}
```

この宣言文を書けば、”std::”を書く必要がない

[注]

- 名前衝突を避けるため、この宣言は決して**ブロックの外**で使わないこと
- ただし、この講義では、コードの可読性のため使う

自習課題

- 本節のトピックス内容の理解を深めるため、講習会用に配布した「実習用ソースコード」の中に自習プログラムが用意されている
- 以下に示す自習プログラム ディレクトリに行き、'AAA_Instruction.txt' ファイルを読み、そこに書かれた指示に従って、自習を行う

自習プログラム ディレクトリ:

- [Geant4Tutorial_20xx/P03_C++ForG4Users_Homework/C01_HelloWorld](#)

オブジェクトとは？

オブジェクトとは何か？

■ オブジェクトとは従来のプログラムでの「変数」を拡張したもの

- 変数: メモリ領域にデータを保持
- オブジェクト: メモリ領域にデータと操作関数をまとめて保持

以下の定義文は二通りに読める:

```
int i = 10;
```

変数*i*は"int"タイプを持ち、データ内容は"10"

オブジェクト*i*は"int"タイプを持ち、データ内容は"10"

- ◆ すなわち、*i*は単にメモリ領域の名前で、命名に変数とオブジェクトの区別はない
- ◆ *i*が専有するメモリ領域のサイズは、タイプで決まる ← タイプと型は同義 (次スライド参照)
- ◆ オブジェクトの概念を理解する時、変数と同じものと考えてOK

■ オブジェクトはなぜ必要か

- 情報(データ)は内容操作に明示的ルール設定がないと、信憑性が保証されない
- オブジェクトは情報(データ)と、それを操作するルールを定めた操作関数が一体となったもの
- 情報(データ)への任意アクセスを禁止することで、内容の信憑性を保証 (データ隠蔽とよぶ)
- オブジェクトの概念は大規模なプログラムを設計、開発するには不可欠 (オブジェクト指向設計)
- Geant4では、シミュレーションに必要な要素(粒子、物質、等)全てオブジェクトにマップされる

オブジェクトのタイプ

■ タイプとは

- **タイプ** は変数の整数**型**、浮動小数点**型**、などの「**型**」と同じ意味
- オブジェクトは**タイプ**を指定で、データの占有メモリ量が決まる
- また、オブジェクトが持つ**操作関数**が決定される
- C++ではオブジェクト作成、関数引数・返り値の指定に**タイプ**を指定が必須
- C++は多数の**タイプ**を標準で提供 — また、ユーザは**独自のタイプ**を定義できる(以下の**クラス**参照)

■ C++が標準で提供する**タイプ**の例

[built-in**タイプ**の例]

タイプ	データ	操作関数
int	整数	+, -, *, / ,
float	小数(単精度)	同上
double	小数(倍精度)	同上
char	文字	同上

[標準ライブラリ提供**タイプ**の例]

タイプ	データ	操作関数
vector	複数のオブジェクト	size, at,
array	複数のオブジェクト	at, size, swap,
complex	複素数	real, imag, +, -, *, / ,
string	文字列	find, substring,

オブジェクトのデータ設定とアクセス

■ オブジェクトへのデータ設定（初期化）

```
int i = 10;
float f( 12.34 );
double g{ 56.78 };

vector<int> j( 1, 2, 3 );
vector<double> v{ 3.4, 4.5 };
complex<float> c( 7.8f, 8.9f );
```

← Built-inタイプのオブジェクト初期化

- `= ( ) { }` が使える: `{ }` を使うのが modern C++ では標準的

← 標準ライブラリ提供タイプのオブジェクト初期化

- `<...>` はテンプレートといわれ、`<>` 内にタイプをパラメータとして指定
- `vector<int>` は整数を、`vector<double>` は小数を保持する vector
- `<>` にでくわしたら、その中にタイプを書けばよいと記憶するので OK

■ オブジェクトのデータへのアクセス

```
cout << i << " , " << f << " , "
    << g << endl;

double d = v.at(1);
int l = v.size();
float r = c.real();

cout << d << endl;
cout << l << endl;
cout << r << endl;
```

← Built-inタイプのオブジェクト・データにアクセス

- オブジェクト名 (`i`、`f`、`g`) を指定する
- `cout` 実行の出力: 10, 12.34, 56.78

← 標準ライブラリタイプのオブジェクト・データにアクセス

- オブジェクトが持つ操作関数 `at`、`size`、`real` 等を使用
- 操作関数はオブジェクト名の後に、`'.'` 演算子を用いて指定
- `cout` 実行の出力(`d`): 4.5
cout 実行の出力(`l`): 2
cout 実行の出力(`r`): 7.8

オブジェクトのポインターとレファレンス

■ オブジェクトのポインターとレファレンスとは何か

- オブジェクトに間接アクセスする手段を提供
- オブジェクトに直接アクセスできるのに、なぜポインター / リファレンスを使い間接アクセスが必要か？
 - ◆ new演算子(後述)を使うときにはポインターが不可欠
 - ◆ オブジェクトの受渡し効率と関係 (後述)

■ ポインターとは何か

- 任意のオブジェクトのメモリ・アドレス情報を保持
- オブジェクトのメモリ・アドレス情報を取得するにはオブジェクトに'&'をつける

```
float f = 1.234;
```

```
float* pf = &f;
```

```
float g = *pf;
```

‘f’ オブジェクトのメモリアドレスを‘&演算子’を使い ‘pf’ に代入

- ‘pf’は‘f’ へのポインター
- ‘float*’ はfloatオブジェクトを指すポインター・タイプ(型)を意味する

ポインター ‘pf’ から、それが指すオブジェクトを‘*演算子’を使い ‘g’ に代入

- ‘dereferencing’ とよばれる: ‘g’は‘f’ と同じ値を持つ
- ‘*演算子’の‘*’と、‘float*’の‘*’とは無関係で別もの

■ レファレンスとは何か

- 任意のオブジェクトに付けた別名で、それ自体はオブジェクトではない (C++独特の仕様)
- リファレンスを使うことは、その元のオブジェクトの名前を使うのと同じこと
- リファレンスはリファレンスするオブジェクトのタイプに‘&’をつける

```
float f = 1.234;
```

```
float& rf = f;
```

‘rf’ は ‘f’ オブジェクトのリファレンス

- リファレンスでの‘&’と、ポインタ取得用の‘&演算子’の‘&’とは無関係で別もの

自習課題

- 本節のトピックス内容の理解を深めるため、講習会用に配布した「実習用ソースコード」の中に自習プログラムが用意されている
- 以下に示す自習プログラム ディレクトリに行き、'AAA_Instruction.txt' ファイルを読み、そこに書かれた指示に従って、自習を行う

自習プログラム ディレクトリ:

- `Geant4Tutorial_20xx/P03_C++ForG4Users_Homework/C02_WhatIsObject`

クラスとは？

クラスとは何か？

■ クラスはユーザが独自のタイプを作る時に使用するツール

- クラスはタイプ(型)と同義語
- クラスを定義することで、C++標準が提供しないタイプ(ユーザ定義タイプ)を作れる
- 大規模プログラム作成には、クラスを基礎とする設計が重要
- Geant4は多数のクラスで設計されている
- Geant4ユーザーは提供されるクラスからオブジェクトを作り、シミュレーションを完成させる

■ クラス定義の具体例

- Geant4では、物体の記述は直方体、円錐などの立体要素をクラスで表現/定義する
- 以下にクラス定義の単純例として、2次元図形(長方形)のクラス表現を示す (対応するC++コードは次スライド)

クラス名: Rectangle	←	長方形
保持するデータ		
◆ sizeX	←	x辺の長さ
◆ sizeY	←	y辺の長さ
保持する操作関数		
◆ Constructor()	←	オブジェクト生成関数
◆ area()	←	面積値を返す関数

[注]

以後の講義では、C++で通常用いられる言葉を使う:

保持するデータ	→	データメンバー
保持する操作関数	→	メンバー関数

なお、メンバーは、クラスのデータと関数の総称

Rectangleクラスのコード

```
#include <iostream>
using namespace std;
```

```
class Rectangle {
```

```
public:
```

```
    Rectangle( double x, double y ) {
        sizeX = x;
        sizeY = y;
    }
```

```
    double area() {
        return sizeX*sizeY;
    }
```

```
private:
```

```
    double sizeX;
    double sizeY;
};
```

コード環境の設定

- 使用するライブラリのinclude
- 標準ライブラリ使用時に “std:.”は省略できる

Class宣言

- `Rectangle`タイプを定義する

public宣言

- この宣言のもとで定義されるメンバーは、ユーザがプログラム中で使用可能

コンストラクタ関数の定義

- `Rectangle`クラスのオブジェクト生成する関数
- 関数名は必ずクラス名と同じ
- 関数引数 (x,y) から渡された値を`sizeX`, `sizeY`に設定
- `コンストラクタ`は次の節で詳しく説明

メンバー関数`area()`を定義

- 帰り値は長方形の面積

private宣言

- この宣言のもとで定義されるメンバーは、ユーザがプログラム中で使用不可
- クラスの実装者のみ可以使用

データメンバー

- `sizeX`, `sizeY`は長方形の2辺のサイズ
- データ隠蔽されている

Rectangleクラスの使用例

■ ヘッダーファイルと実装ファイル

- クラスは実用上、**ヘッダ・ファイル(.hh)**と**実装・ファイル(.cc)**に分けて定義する（拡張子名はGeant4準拠）
- ユーザはヘッダファイルを**"include"**するだけで、クラスが使用可能（実装はブラックボックスとなる）

ヘッダファイル (Rectangle.hh)

```
class Rectangle {  
public:  
    Rectangle( double x, double y );  
    double area();  
  
private:  
    double sizeX;  
    double sizeY;  
};
```

実装ファイル (Rectangle.cc)

```
#include "Rectangle.hh"  
  
Rectangle::Rectangle( double x, double y ){  
    sizeX = x;  
    sizeY = y;  
}  
  
double Rectangle::area() {  
    return sizeX*sizeY;  
}
```

[注] 実装ファイルで定義されているRectangle(), area() 関数は**スコープ解決演算子 '::'**により、Rectangleクラスのメンバー関数であると指定される

■ main.cc ファイル

Mainプログラムファイル (main.cc)

```
#include <iostream>  
using namespace std;  
#include "Rectangle.hh"  
  
int main() {  
    Rectangle rect{ 3.0, 4.0 };  
    cout << "Area: " << rect.area() << endl;  
}
```

← **ヘッダファイル**をincludeするだけで、Rectangleクラスのメンバーを使用することができる

← Rectangleのオブジェクト'rect'を生成

← オブジェクト'rect'の**メンバー関数**area()を使う

- メンバー関数へのアクセスは**ドット演算子(.)**を使用
- main()を実行すると端末に"Area: 12"と出力される

[注] 実行プログラムの作成手法は自習課題を参照

自習課題

- 本節のトピックス内容の理解を深めるため、講習会用に配布した「実習用ソースコード」の中に自習プログラムが用意されている
- 以下に示す自習プログラム ディレクトリに行き、'AAA_Instruction.txt' ファイルを読み、そこに書かれた指示に従って、自習を行う

自習プログラム ディレクトリ:

- [Geant4Tutorial_20xx/P03_C++ForG4Users_Homework/C03_WhatIsClass](#)

オブジェクト生成と存続期間

コンストラクタとデストラクタ — クラスのオブジェクト生成と消滅

■ コンストラクタ

- コンストラクタはクラスオブジェクトを作る時に使うpublicなメンバー関数
- コンストラクタ関数名はクラス名と必ず同一
- コンストラクタは通常複数存在する（関数名は全て同じ：関数の引数の型で区別）
- コンストラクタ関数は帰値(return)を持たない

[注] 一般的に、引数の型が異なれば、同名の関数を複数定義可能

→ 関数多重定義(Overloading)

■ デストラクタ

- デストラクタは生成されたオブジェクトが消滅する時によばれるpublicなメンバー関数
- デストラクタ関数名はクラス名にチルダ'~'が付いている

■ コンストラクタとデストラクタの例 — 前節のRectangleクラスを変更

```
class Rectangle {  
public:  
    Rectangle( double x, double y );  
    Rectangle( double xy );  
    Rectangle( );  
  
    ~Rectangle( );  
};
```

← Rectangleの長方形の(x, y)を設定するコンストラクタ

← Rectangleの正方形として設定するコンストラクタ

← Rectangleの長方形の(x, y)を設定しないコンストラクタ

← Rectangleのデストラクタ

[注] new演算子を用いてコンストラクタで作ったオブジェクトは、必ずデストラクタで消滅させないと、メモリリークが生じる

← この問題は次のスライドの「オブジェクトの存在期間」で扱う

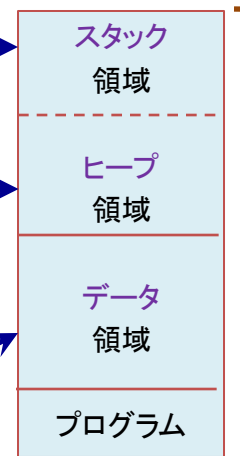
オブジェクトの存続期間

■ オブジェクトの存続期間(寿命)

- オブジェクトのプログラム内での存続期間は、オブジェクトがどのように作られたかで決まる
- オブジェクトの存続期間は以下の3種類に分類
 - それぞれプログラムのメモリ空間の異なる領域に保存される

1. **ローカル・オブジェクト**: プログラムのスタック領域に保存
 - **ブロック内**で作られ、**ブロック内**から出ると消滅
(関数定義等で使われる中括弧 `{ }` 内を、「**ブロック**」とよぶ)
2. **ヒープ・オブジェクト**: プログラムのヒープ領域に保存
 - **new**演算子を使用し作成、プログラム終了時まで存在
 - **ブロック内**のオブジェクトを消滅させない手段
 - **new**はヒープに作られたオブジェクトの**ポインター**を返す
3. **グローバル・オブジェクト**: プログラムのデータ領域に保存
 - **ブロック外**で作られ、プログラム終了時まで存在

プログラム
メモリ空間



マップ

計算機
物理メモリ

0x00001234
0x00001235
0x00001236
.....
.....
0x00100000
0x00100001
0x00100002
.....
.....
.....
.....

■ 異なる存続期間のオブジェクト作成例

```
const float pi = 3.14;

int main() {
    double g{ 56.78 };

    auto pVec = new vector<float>{ 1.2, 2.3 };

    delete pVec;
}
```

← `{ }`外で作られた`pi`: グローバル・オブジェクト

▷ 'データ領域'に保存、プログラム終了時まで存在

← `{ }`内で作られた`g`: ローカル・オブジェクト

▷ 'スタック領域'に保存、`{ }`を出ると消滅

← `new`演算子で作られた`pVec`: ヒープ・オブジェクト

▷ 'ヒープ領域'に保存、プログラム終了時まで存在

▷ `pVec`は保存されたオブジェクトのポインター

▷ `auto`で自動的にオブジェクト型を推論してくれる

▷ `delete`演算子で消去しないとメモリ・リーク発生

関数でのオブジェクトの受け渡し

■ オブジェクトの受け渡し

- メンバー関数がオブジェクトを受け取り、それを処理し結果をオブジェクトとし返すことはよくある
- 関数ブロック中のオブジェクトをブロック外に受け渡しするには、コピーが必要
 - ◆ ブロック中のオブジェクトはローカル・オブジェクト
- 大サイズのオブジェクトを扱う時は、コピー速度がプログラム全体の処理性能に大きく影響
 - ◆ ユーザはオブジェクトのコピー時には、常に速度に注意を払う必要
- 関数のオブジェクト受け渡しの方法には次に述べる4手法がある

■ オブジェクトの受け渡し手法と速度

- 1) オブジェクト渡し [低速] オブジェクト自体を渡す

```
void PassValue( float val ) // 元のvalは関数実行後、不変
```

- 2) ポインター渡し [高速] オブジェクト・ポインタを渡す

```
void PassPoint( float* val ) // 元のvalは関数実行後、変化
```

- 3) レファレンス渡し [高速] オブジェクト・レファレンスを渡す

```
void PassReference( float& val ) // 元のvalは関数実行後、変化
```

- 4) 返り値渡し [高速] オブジェクトをreturnで渡す (RVO:Return Value Optimization: c++11以降)

```
float PassReturn( float val ) // 元のvalは関数実行後、変化 (return値をvalにする)
```

[注] 具体的な使用例は自習課題'C04_ObjectLifetime'を参照

自習課題

- 本節のトピックス内容の理解を深めるため、講習会用に配布した「実習用ソースコード」の中に自習プログラムが用意されている
- 以下に示す自習プログラム ディレクトリに行き、'AAA_Instruction.txt' ファイルを読み、そこに書かれた指示に従って、自習を行う

自習プログラム ディレクトリ:

- [Geant4Tutorial_20xx/P03_C++ForG4Users_Homework/C04_ObjectLifetime](#)

クラス継承と仮想関数

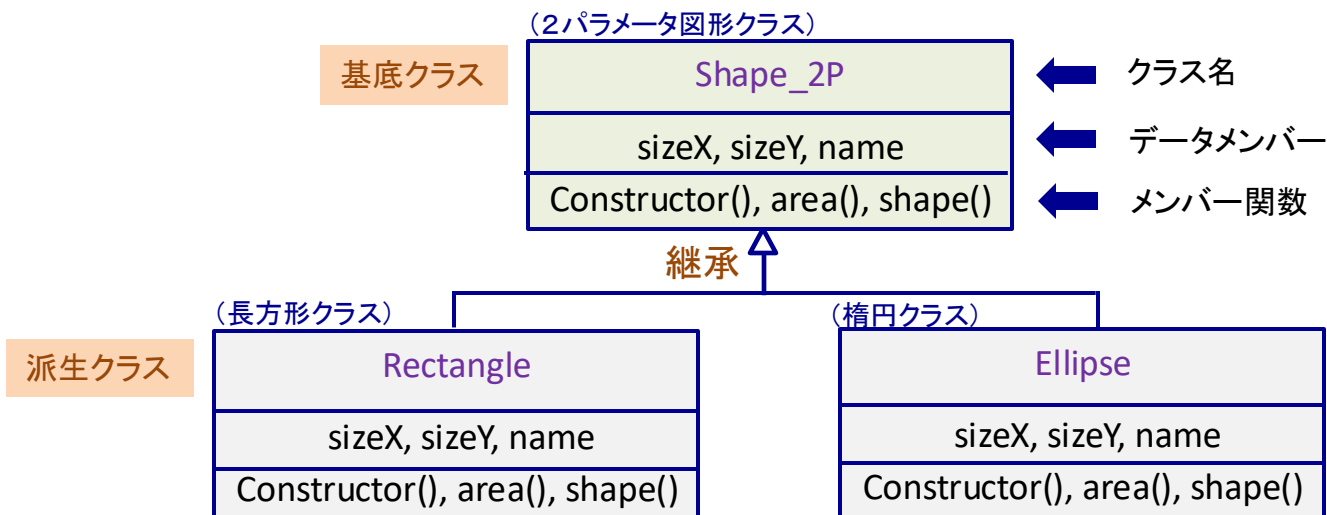
クラス継承

■ クラス継承とは何か

- 既存クラスをもとに、その機能を拡張、変更等を行い、新たなクラスを作ること
- もとのクラスを基底クラス、新しく作るクラスを派生クラスとよぶ
- Geant4がツールとしてユーザに提供するクラスは、内部的なクラスを継承し作られている

■ クラス継承の具体例

- 以下にクラス継承の単純例として、以下の基底クラスと派生クラスの表現を示す
 - ◆ 基底クラス: 2つのパラメータで表現できる図形のクラス
 - ◆ 派生クラス: 長方形クラス、楕円クラス



[注]

派生クラスは基底クラスのメンバー全てを共有している

クラス継承のコード

■ 前スライドで示したクラス継承のコードを以下に示す

```
class Shape_2P {  
public:  
    string shape(){ return name; }  
    double area() { return 0.; }  
protected:  
    double sizeX, sizeY;  
    string name = "Shape_2P";  
    const double pi = 3.14159265;  
};
```

基底クラス Shape_2Pを定義

- コンストラクタ定義がないと、C++が自動的にdefaultを作成

派生クラスはこれらの全メンバーを共有する

- public宣言されたメンバーは派生クラスでもpublic
- protected宣言されたメンバーは派生クラスだけがアクセス可能
- 宣言なしメンバーはprivate宣言と同一：派生クラスからアクセス不可

```
class Rectangle : public Shape_2P {  
public:  
    Rectangle ( double x, double y ) {  
        sizeX = x; sizeY = y;  
        name = "Rectangle";  
    }  
    double area(){ return sizeX * sizeY; }  
};
```

派生クラス Rectangleを定義

- ' :public'は基底クラスをpublicで継承
- 基底クラスのprotectedメンバーにアクセス可能

Rectangleのコンストラクタを定義

- 'name'が長方形に一致するように再定義

area関数が長方形に一致するように再定義

- 関数オーバーライドとよぶ

```
class Ellipse : public Shape_2P {  
public:  
    Ellipse( double x, double y ) {  
        sizeX = x; sizeY = y;  
        name = "Ellipse";  
    }  
    double area(){ return sizeX * sizeY * pi/4.0; }  
};
```

派生クラス Ellipseを定義

- ' :public'は基底クラスをpublicで継承
- 基底クラスのprotectedメンバーにアクセス可能

Ellipseのコンストラクタを定義

- 'name'が楕円形に一致するように再定義

area関数が楕円形に一致するように再定義

- 関数オーバーライドとよぶ

Shape_2P/Rectangle/Ellipseクラスの使用

■ Shape_2P/Rectangle/Ellipseクラスの使用コード例

```
#include <iostream>
#include <string>
#include "Shape_2P.hh"
#include "Rectangle.hh"
#include "Ellipse.hh"
using namespace std;

int main() {

    Shape_2P pict{ };
    cout << "Shape: " << pict.shape() << endl;
    cout << "Area: " << pict.area() << endl;

    Rectangle rect{3.0, 4.0};
    cout << "Shape: " << rect.shape() << endl;
    cout << "Area: " << rect.area() << endl;

    Ellipse elps{5.0, 6.0};
    cout << "Shape: " << elps.shape() << endl;
    cout << "Area: " << elps.area() << endl;
}
```

Mainプログラムファイル (main.cc)

← ヘッダーファイルを"include"して3つのクラスを使う

[注] 前スライドではクラスはヘッダーと実装ファイルに分離されていない: ここでは分離されていると仮定
(ヘッダーと実装に分けたコードは自習課題を参照)

← Shape_2Pオブジェクト(pict)を生成

← pictのメソッドアクセスはドット演算子(.) 使用

← Rectangleオブジェクト(rect)を生成

← rectのメソッドアクセスはドット演算子(.) 使用

← Ellipseオブジェクト(elps)を生成

← elpsのメソッドアクセスはドット演算子(.) 使用

```
Shape: Shape_2P
Area: 0
Shape: Rectangle
Area: 12
Shape: Ellipse
Area: 23.5619
```

← 上のプログラムの実行結果

[注] 実行プログラムの作成手法は自習課題を参照

仮想関数

■ 仮想関数とは何か

- メンバー関数は通常、コンパイル時に振る舞いが**静的に決定**(Static Binding)される
- メンバー関数の振る舞いをプログラム実行時に**動的に決定**(Dynamic binding)したい場合がある
 - ◆ [例] “2パラメータ図形”が長方形或いは楕円の場合、area()関数の振る舞いを自動的に切り替る
 - ◆ メンバー関数が動的に振る舞いを変化させることを**動的多態性**(polymorphism)とよぶ
 - ◆ Geant4で**動的多態性**(polymorphism)はカーネル設計の根幹 (例: 種々の物理相互作用を統一的に扱う)
- 基底クラスのメンバー関数を**仮想関数**と定義する:
 - ◆ 関数の振る舞いはコンパイル時には静的に決定されない
 - ◆ プログラム実行時に、関数にアクセスしたオブジェクトによりが持つ振る舞いが**動的に決定**される

[注] Geant4ユーザは**動的多態性**を直接扱う必要はない。しかし、カーネル構造を理解したい時には必要な知識

■ 仮想関数の作成

- メンバー関数を'**virtual**'宣言すれば**仮想関数**となる
- 前演習の'**Shape_2P**'基底クラスで定義された'**area()**'関数を**仮想関数**に指定

virtual宣言により
area()は**仮想関数**になる

```
Shape_2P.cc
class Shape_2P {
public:
    string shape(){ return name; }
    virtual double area() { return 0.; }
protected:
    double sizeX, sizeY;
    string name = "Shape_2P";
    const double pi = 3.14159265;
};
```

[注] 派生クラスで仮想関数を再定義する際、**override**キーワードを使い、再定義可能かどうかをcompilerに確認させることを推奨
— 使用例はGeant4のhands-onに出てくる

仮想関数の使用例

■ 仮想関数で動的多態性を実現するキーポイント

- 基底クラスのオブジェクトのポインターは、常に派生クラスのオブジェクトをポイントすることができる
- 基底クラスのオブジェクトのポインターのポイントする派生クラスのオブジェクトを動的に切り替える

■ 仮想関数の使用コード例

- 前スライドで示した'Shape_2P'基底クラス(virtual指定あり)を使用
- Rectangle/Ellipseは「継承」スライドのものを、そのまま使用できる

```
#include <iostream>
#include <string>
#include "Shape_2P.hh"
#include "Rectangle.hh"
#include "Ellipse.hh"
using namespace std;

int main() {
    Shape_2P* pShape;
    Rectangle rect{3.0, 4.0};
    Ellipse elps {5.0, 6.0};

    pShape = &rect;
    cout << "Shape: " << pShape->shape() << endl;
    cout << "Area: " << pShape->area() << endl;

    pShape = &elps;
    cout << "Shape: " << pShape->shape() << endl;
    cout << "Area: " << pShape->area() << endl;
}
```

プログラムの実行結果

Shape: Rectangle
Area: 12
Shape: Ellipse
Area: 23.5619

[注]

virtual指定なしでプログラムを実行すると動的多態性は作動せず、結果はAreaが全て0

動的多態性を制御するポインター

動的多態性をで切り替えられるオブジェクト

Shape_2Pポインタをrectへのポインタに動的設定

pShapeが指すオブジェクト・メンバーにアクセス

- rectのメンバーにアクセスすることになる
- ポインタ経由のメンバーアクセスは演算子'-'>'を使う

Shape_2Pポインタをelpsのポインタに動的設定

pShapeが指すオブジェクト・メンバーにアクセス

- elpsのメンバーにアクセスすることになる

純粋仮想関数

■ 純粋仮想関数(pure virtual function) とは

- 以下のように'virtual'が宣言され、さらに' = 0 'が設定された関数

```
virtual double area() = 0;
```

- 上の宣言は、area()が未定義で、ユーザの実装が不可欠であることを意味する
- クラスが純粋仮想関数を持つと、そのクラスは抽象クラスとよばれる
- 抽象クラスからの派生クラスは、必ず純粋仮想関数を実装しなければならない
- 抽象クラスは自分のオブジェクトを作れない

[注] 前スライドの「仮想関数の使用例」ではarea()関数は純粋仮想関数でないので、Shape_2Pのオブジェクトを'new'で作った。もし、純粋関数と設定したら、Shape_2P'のオブジェクトは作れない。

■ Geant4と純粋仮想関数

- Geant4は、ユーザが必ず実装しなければならないクラスに純粋仮想関数を設定している
- Geant4ユーザはライブラリ使用にあたり、純粋仮想関数に遭遇したらその実装が必須

自習課題

- 本節のトピックス内容の理解を深めるため、講習会用に配布した「実習用ソースコード」の中に自習プログラムが用意されている
- 以下に示す自習プログラム ディレクトリに行き、'AAA_Instruction.txt' ファイルを読み、そこに書かれた指示に従って、自習を行う

自習プログラム ディレクトリ:

- `Geant4Tutorial_20xx/P03_C++ForG4Users_Homework/C05_Inheritance_VirtualFunction`

講義 終了